GEMSTONE / PHARO

GemStone-Pharo-Bridge User Manual

Practical guide for installing the bridge, connecting to GemStone, and using the Session Browser, GemStone Browser, Workspace, Symbol List Browser, Remote Debugger, connectors, and support tools.

Repository: `unicompute/GemStone-Pharo-Bridge`
Prepared from codebase state: 2026-04-19
Primary baseline: `BaselineOfGemStonePharo`
Audience: Pharo users working with GemStone sessions and GemStone-side tools

> **Scope.** This manual is based on the current user-facing tools in the repository: load groups, session management, GemStone browsing, workspace execution, transcript capture, Symbol List inspection, remote debugging, and connector administration.

Contents

- 1. What the bridge provides
- 2. Prerequisites
- 3. Installing and loading with Metacello
- 4. Loading with Iceberg
- 5. Starting the tools
- 6. Session Browser workflow
- 7. GemStone Browser workflow
- 8. GemStone Workspace workflow
- 9. Symbol List Browser workflow
- 10. Remote Debugger workflow
- 11. Connector Browser and connector maintenance
- 12. Transcript, utilities, and admin tools
- 13. Testing and regression scripts
- 14. Troubleshooting
- 15. Quick reference

1. What the bridge provides

GemStone-Pharo-Bridge gives Pharo a user-facing toolset for logging into GemStone and working with GemStone objects and code from a Pharo UI. The codebase is split into a Smalltalk-first core and optional MagLev/Ruby extension packages.

Load group	What it contains	Typical use
`Original`	`GemStone-GBS-Converted`, `GemStone-GBS-Tools`	original/base production packages only
`Original-Tests`	`GemStone-GBS-Converted`, `GemStone-GBS-Tools`, `GemStone-Pharo-Tests`	original/base production packages plus the original/base test layer
`Core`	`GemStone-GBS-Converted`, `GemStone-GBS-Core`	Smalltalk core bridge only
`Core-Tests`	`GemStone-GBS-Converted`, `GemStone-GBS-Core`, `GemStone-GBS-Tools`, `GemStone-GBS-Core-Tools`, `GemStone-Pharo-Core-Tests`	Smalltalk core, generic tool overlays, and the core-only test suite
`MagLev`	`GemStone-GBS-Converted`, `GemStone-GBS-Core`, `GemStone-GBS-Tools`, `GemStone-GBS-Core-Tools`, `GemStone-GBS-MagLev`, `GemStone-GBS-MagLev-Tools`	convenience alias for the MagLev production stack

Load group	What it contains	Typical use
`Tests`	`GemStone-Pharo-Tests`, `GemStone-Pharo-Core-Tests`, `GemStone-Pharo-MagLev-Tests`	convenience alias for all active core and MagLev-aware test packages
`default`	`GemStone-GBS-Converted`, `GemStone-GBS-Core`, `GemStone-GBS-MagLev`, `GemStone-GBS-Tools`, `GemStone-GBS-Core-Tools`, `GemStone-GBS-MagLev-Tools`, `GemStone-Pharo-Tests`, `GemStone-Pharo-Core-Tests`, `GemStone-Pharo-MagLev-Tests`	alias for `Full`

A generated switch matrix is also available at `doc/LOAD-MATRIX.md` for the base/original, core, and MagLev load combinations.

For the practical loading and usage guide, including local disk and GitHub Metacello snippets plus the classic session example, see `doc/MAGLEV-BRANCH-USAGE.md`.

The active core root API is `bridgeRoot`, backed by `#GbsBridgeRoot` and `GbsBridgeRootFacade`.

The bridge centers around these user tools:

- **Session Browser:** manage connection profiles and active sessions
- **GemStone Browser:** browse dictionaries, classes, protocols, methods, and source
- **GemStone Workspace:** evaluate GemStone code, print results, inspect values, and open the debugger on errors
- **Symbol List Browser:** inspect and edit symbol dictionaries, including `UserGlobals`
- **Remote Debugger:** step, inspect, and recompile against suspended GemStone contexts
- **Connector Browser:** inspect and configure connector behavior
- **Remote Transcript:** collect GemStone transcript output in a Pharo window

2. Prerequisites

Before loading the bridge, make sure you have the following in place:

- A Pharo image with Metacello available
- A reachable GemStone stone and the corresponding GCI client libraries available to Pharo
- A valid GemStone user account and password
- If using a private GitHub repository, working GitHub credentials for Iceberg or Git

Expected connection fields

Session profiles support the usual connection settings and a set of advanced fields. The current bridge initializes new profiles with a default stone name of `gs64stone` and a default service of `gemnetobject`.

- Profile name
- Stone name
- Username and password

- Gem service
- Optional X509 certificate fields
- Optional NetLDI host/IP and port/name fields
- Optional extra gem arguments and log settings

3. Installing and loading with Metacello

Recommended local-path load

If the repository is already cloned locally, loading from the local Tonel repository is the simplest path.

```
Metacello new
  baseline: 'GemStonePharo';
  repository: 'tonel:///path/to/GemStone-Pharo-Bridge/src';
  load: #('Core-Tests').
```

For the full interactive toolset, including MagLev/Ruby extensions when present:

```
Metacello new
  baseline: 'GemStonePharo';
  repository: 'tonel:///path/to/GemStone-Pharo-Bridge/src';
  load.
```

Available load groups

```
"Runtime only"
Metacello new
  baseline: 'GemStonePharo';
  repository: 'tonel:///path/to/GemStone-Pharo-Bridge/src';
  load: #('Core').

"Smalltalk core plus generic tools and Smalltalk-only tests"
Metacello new
  baseline: 'GemStonePharo';
  repository: 'tonel:///path/to/GemStone-Pharo-Bridge/src';
  load: #('Core-Tests').

"Compatibility-only alias coverage"
Metacello new
  baseline: 'GemStonePharo';
  repository: 'tonel:///path/to/GemStone-Pharo-Bridge/src';
  load: #('Compatibility-Tests').

"Optional MagLev/Ruby layer"
Metacello new
  baseline: 'GemStonePharo';
  repository: 'tonel:///path/to/GemStone-Pharo-Bridge/src';
  load: #('MagLev').
```

```
"Full developer load"
Metacello new
  baseline: 'GemStonePharo';
  repository: 'tonel:///path/to/GemStone-Pharo-Bridge/src';
  load.
```

What postload does

The baseline runs pre-load and post-load hooks that remove and re-apply runtime and tools overrides. If you are reloading the bridge during development, this is expected behavior.

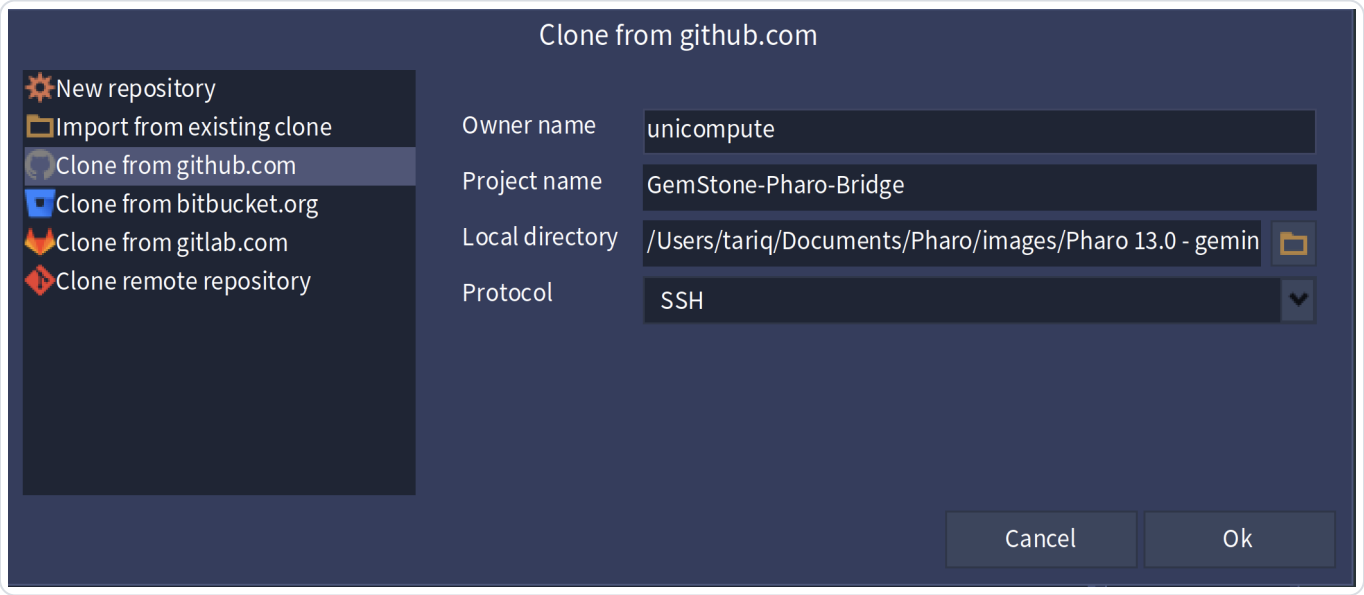
4. Loading with Iceberg

Iceberg is a good fit for this project. The repository already contains the metadata Iceberg expects for a Tonel project:

- `.project` with `srcDirectory = src`
- `src/.properties` with `#format : #tonel`

Basic Iceberg workflow

1. In Pharo, open Iceberg.
2. Add the repository from GitHub or clone it locally first.
3. Check out the branch you want.
4. Load the baseline or evaluate the Metacello expression against the local checkout.



Existing repository screenshot shipped in `doc/iceberg.png`.

Private GitHub repositories

If the repository is private, Iceberg still needs valid GitHub credentials. In practice that usually means:

- an SSH remote plus a loaded SSH key, or
- an HTTPS remote plus a GitHub personal access token

> **Recommendation.** For private repositories, cloning with Iceberg and then loading from the local Tonel path is usually more predictable than trying to make Metacello itself authenticate directly against GitHub.

5. Starting the tools

After loading a tools-capable group such as `Core-Tests`, `MagLev`, or `default`, open the tools from the World menu:

```
World menu → GemStone
```

The current menu structure is:

- **Sessions** → Session Browser, Simple Launcher, Logout Current Session
- **Connectors** → Connector Browser and connector maintenance commands
- **Browse** → All Classes, Class, Namespace, Symbol List Browser, Senders, Implementors, hierarchy queries, references, substring search
- **Admin** → Users, Namespaces, Security Policies
- **Tools** → New GS Workspace, GS File in, Settings, Breakpoints, System Workspace
- **Utilities** → supporting tools such as file logging and diagnostics

Useful open expressions

```
GbsSessionBrowserPresenter new open.
GbsWorkspace new open.
GbsRemoteTranscript new open.
```

6. Session Browser workflow

The Session Browser is the normal starting point. It shows two lists: a parameter list of saved connection profiles and a session list of active or remembered sessions.

Primary actions

Control	Purpose
`Add...`	Create a new connection profile
`Copy`	Duplicate a selected profile
`Edit...`	Edit the selected profile
`Delete`	Remove the selected profile
`Login Link` / `Login RPC`	Log in using the selected profile
`Current`	Mark the selected session as the current session
`Commit`	Commit the current transaction
`Abort`	Abort the current transaction
`Begin`	Begin a GemStone transaction
`Break`	Issue a break command to the session
`Logout`	Log out the selected session

Typical login sequence

1. Open **GemStone** → **Sessions** → **Session Browser**.
2. Click `Add...` and create a connection profile.
3. Enter at least the profile name, stone name, username, password, and gem service.
4. Select the profile and click `Login RPC`.
5. Once the session appears as connected, click `Current`.
6. Double-click the session row to open a GemStone Workspace.

Current session matters

The Workspace, Browser, Transcript, and many menu commands operate on `GbsSessionParameters currentSession`. If a tool appears to use the wrong session, set the intended session current in the Session Browser and try again.

7. GemStone Browser workflow

The GemStone Browser is the main code browser for server-side GemStone classes and methods. It opens on the current connected session and displays:

- symbol dictionaries
- classes
- protocols
- methods
- source text

What you can do in the browser

- browse all classes or a specific class
- open a namespace/dictionary view
- query senders, implementors, hierarchy senders, hierarchy implementors, references, and methods containing a substring
- add dictionaries, classes, categories, instance variables, class variables, and class instance variables
- edit and compile methods
- optionally use auto-commit after changes

Practical use

1. Choose **GemStone** → **Browse** → **All Classes** or **Class...**.
2. Select a dictionary, then a class.
3. Select a protocol and method to view its source.
4. Edit the source and accept it to compile back to GemStone.
5. Commit manually unless auto-commit is enabled.

> **Note.** The browser is working against the GemStone symbol list visible to the logged-in user. That means dictionary visibility and write permissions still depend on the connected GemStone account.

8. GemStone Workspace workflow

The GemStone Workspace is the easiest place to evaluate code against the current session. It is optimized for one-off execution, print-it style evaluation, inspection, commit, abort, and debugger entry on errors.

Main commands

Action	Effect
`Execute in GemStone`	Runs the selection or entire workspace in GemStone
`Commit`	Commit the current transaction
`Abort (Sync)`	Abort changes and sync with the latest GemStone state

Action	Effect
`Print It`	Appends the printed result into the editor
`Inspect It`	Inspects the resulting remote object
`GS File It In`	Executes the selected source as a GemStone file-in

Shortcuts

- `Cmd+G` or `Ctrl+G`: execute in GemStone
- `Cmd+D`: do it
- `Cmd+P`: print it

Example

```
1 + 1
"should execute successfully"

1 / 0
"should open the GemStone remote debugger"
```

When execution raises a `GbsError` with remote process metadata, the workspace opens the remote debugger automatically instead of only showing an alert.

9. Symbol List Browser workflow

The Symbol List Browser is the right tool for working with symbol dictionaries and globals such as `UserGlobals`. It is distinct from the regular browser:

- The regular GemStone Browser focuses on classes and methods.
- The Symbol List Browser focuses on dictionaries and entries.

What it shows

- a target user selector
- that user's dictionaries
- entries in the selected dictionary
- a bottom preview table that renders dictionaries as key/value rows and collections as index/value rows

Typical uses

- browse `UserGlobals` for the current GemStone user
- inspect application state dictionaries
- add or remove dictionaries
- add or remove entries
- inspect nested dictionaries and collections without dropping back to raw `printString`

Inspect Entry behavior

The current build supports structured inspection:

- if the selected entry is a dictionary, it opens a key/value inspector window
- if the selected entry is a collection, it opens an index/value inspector window
- the bottom preview pane also supports inspecting nested values
- for plain scalar values, the browser shows the string representation

When to use it

Use the Symbol List Browser when you need to answer questions like: "What is actually stored in `UserGlobals`?" or "What keys exist in this application dictionary?"

10. Remote Debugger workflow

The Remote Debugger is the bridge's Pharo-side UI for suspended GemStone execution. It is opened from the workspace on errors or explicitly through service code.

What it shows

- GemStone stack frames
- source code for the selected frame
- local variables
- receiver, frame, process, and stack inspection panels

- execution control buttons

Core debugging actions

- step into
- step over
- step return
- resume
- restart
- run to caret
- return from frame
- pause or terminate
- inspect receiver, frame, process, stack, or selected variable

Source editing and recompilation

The debugger also supports accepting source edits. If recompilation succeeds, it refreshes the stack and source view. If recompilation fails, the source pane shows the compile error state.

Manual test cases

The test directory already suggests a useful smoke-test set:

```
1 + 1
1 / 0
| a b | a := 10. b := 0. a / b.
Object thisMessageDoesNotExist
```

> **Expected result.** Simple successful expressions should return normally. Exceptions such as divide-by-zero or message-not-understood should open the remote debugger.

11. Connector Browser and connector maintenance

Connector tools control how local Pharo-side objects and GemStone-side classes are mapped and synchronized. This is an advanced feature, but it is exposed directly from the GemStone menu.

Main connector tasks

- open the Connector Browser for the current session
- initialize fast connectors
- generate class connectors
- remove invalid connectors
- configure connector class mappings
- show or clear connector class overrides

Connector Browser concepts

- **groups**: global or session-specific connector groups
- **connector rows**: client/server class pairs
- **connection state**: connected or disconnected behavior
- **post-connect action**: update strategies and forwarding modes

If you are not actively customizing replication or connector behavior, you can ignore this tool initially. For advanced bridge work, it is the main place to inspect and tune connector mappings.

12. Transcript, utilities, and admin tools

GemStone Remote Transcript

The remote transcript window drains transcript output from the current session and mirrors it into a Pharo window and the local Pharo transcript. This is useful when you want remote logging without switching tools.

- Open it from the launcher's Tools menu or by evaluating `GbsRemoteTranscript new open`.
- Use it to observe output after workspace execution.
- Advanced users can install a GemStone transcript proxy for deeper integration.

Admin tools

The GemStone menu includes direct entry points for:

- **Users**
- **Namespaces**
- **Security Policies**

Use these only with appropriate GemStone permissions. The menu entries assume a connected session and will fail or inform if there is no current login.

Utilities

The bridge also exposes support utilities such as:

- File Logger
- settings dialogs
- stat monitor and tool-specific diagnostics
- breakpoint browser

13. Testing and regression scripts

The repository includes both Smalltalk test packages and shell/Smalltalk scripts for regression workflows.

Steady-state verification lanes

The current bridge keeps a small steady-state verification surface:

- **`core-only`**: Verify the Smalltalk core without optional MagLev production packages and without deleted legacy surface.
- **`bootstrap-smoke`**: Prove that a clean image can micro-bootstrap the helper package and load the requested group before post-load checks run.
- **`original`**: Verify that the original/base production layer reloads cleanly without the generic Core or optional MagLev overlays.
- **`original-drift`**: Verify that the original/base production layer stays clean relative to `56b6db3...`, allowing only the explicit accepted test-layer exceptions.

- **`original-tests`**: Verify the original/base production and original/base test layer without the generic Core or optional MagLev overlays. This lane proves the base unit layer only.
- **`full`**: Verify the steady-state developer load plus the live GemStone lane.
- **`artifact-freshness`**: Verify that the generated contract artifacts and marker-managed doc sections are already up to date.
- **`verify`**: Run core-only, bootstrap-smoke, original, original-drift, original-tests, full, artifact-freshness, then the summary-renderer smoke check.

Expected Original-layer Test Exceptions

`make original-drift` reports the original/base production layer as clean and currently accepts only these test-layer exceptions:

- `src/GemStone-Pharo-Tests/GciError.extension.st`: Adds a tiny test-only GciError number accessor shim so the restored base login-error path can run without production drift.
- `src/GemStone-Pharo-Tests/MockGbsSession.class.st`: Keeps only a narrow interpretLoginError override so the restored base GbsSession production file stays clean.

Generated operator reference

The contract generates a human-facing steady-state operator reference at `doc/USER-MANUAL-REFERENCE.html`. Use that file as the authoritative quick reference for active load groups, verification lanes, live GemStone environment variables, the `bridgeRoot` API, and the no-compatibility proof.

A generated layered switch matrix is available at `doc/LOAD-MATRIX.md`.

Recommended entry point

```
make help
make original-drift
make core-only
make original
make original-tests
make full
make verify
make artifact-freshness
```

Known-good live example

```
GS_USER='DataCurator' \
GS_PASS='swordfish' \
GS_NETLDI_HOST='localhost' \
GS_NETLDI_NAME_OR_PORT='50377' \
GEMSTONE='/Users/tariq/GemStone64Bit3.7.5-arm64.Darwin' \
make full
```

No-compatibility proof

The clean reload lane proves that deleted legacy package, class, selector, and method-package ownership surface stays absent from the image and the active source roots. If that proof fails, the reload lane reports `NO_COMPATIBILITY_PROOF_FAILED`.

Manual debugger smoke tests

The `tests/README.md` file contains a focused manual checklist for the remote debugger. It is a good first validation pass after a fresh load.

Scripts directory

The `scripts` folder contains repeatable automation for tasks such as:

- core-only clean reload checks
- full clean reload and regressions
- contract artifact generation and freshness verification
- live debugger integration runs
- query results presenter tests
- restart and breakpoint scenarios

14. Troubleshooting

No connected GemStone session

Many menu items require a current connected session. If a tool says there is no session:

1. open the Session Browser
2. log in with a saved profile
3. mark the intended session as current
4. retry the tool command

Workspace does not use the session I expect

Set the intended session as current in the Session Browser. The workspace and several browser commands prefer `GbsSessionParameters currentSession`.

Debugger does not open

Not every GemStone-side failure produces walkable remote context metadata. Start with known debugger-opening expressions such as `1 / 0` or a message-not-understood case and verify that you are logged in.

Private GitHub clone problems

If Iceberg can see the repo but cannot fetch it, check your GitHub credentials. For a private repository, the most reliable path is usually:

1. configure SSH or HTTPS token access for Git
2. clone with Iceberg
3. load from the local Tonel path with Metacello

Edits do not appear after reload

Reload the baseline or run the appropriate clean reload script. The bridge applies runtime/tool overrides during load, so partial reloads can leave stale behavior if only some packages are refreshed.

15. Quick reference

Open the main tools

```
GbsSessionBrowserPresenter new open.
GbsWorkspace new open.
GbsRemoteTranscript new open.
```

Load the bridge from a local checkout

```
Metacello new
  baseline: 'GemStonePharo';
  repository: 'tonel:///path/to/GemStone-Pharo-Bridge/src';
  load: #('Core-Tests').
```

Open the Symbol List Browser on the current session

```
GbsClassicLauncher browseSymbolListOnCurrentSession.
```

Open the current UserGlobals

Use **GemStone** → **Browse** → **Symbol List Browser**, choose the current GemStone user, select `UserGlobals`, and then inspect entries from there.

Good first smoke test

1. Log in from the Session Browser
2. Open a GemStone Workspace
3. Evaluate `1 + 1`
4. Evaluate `1 / 0`
5. Open the Symbol List Browser and inspect `UserGlobals`
6. Open the GemStone Browser and browse a class

> This manual is intended to be a working operator guide. For the contract-driven steady-state reference, open `doc/USER-MANUAL-REFERENCE.html`. For deeper implementation detail, read the baseline, presenter classes, and regression scripts in the repository alongside this document.

mplementation detail, read the baseline, presenter classes, and regression scripts in the repository alongside this document.